

Beginner's Terminal & Anaconda Guide

macOS + Windows

What this guide helps you do

Open a terminal, move between folders, create and activate a Conda environment, install project dependencies, run Python programs, start FastAPI/Jupyter, and solve the most common beginner errors.

Designed for learners who are new to command-line development

Keep this PDF beside you while following labs and code demos.

1. Understand the three pieces

Before typing commands, remember this simple mental model:

Terminal / Prompt	The window where you type commands.
Conda / Anaconda	Creates isolated Python environments and helps manage packages.
Python	Runs your .py program inside the active environment.

Recommended beginner setup
macOS: use Terminal. Windows: use Anaconda Prompt for the exercises in this guide. This avoids many PATH and shell differences while you are learning.

2. Basic navigation commands

Task	macOS Terminal	Windows (Anaconda Prompt)
Show current folder	pwd	cd
List files/folders	ls	dir
Enter a folder	cd project_name	cd project_name
Go back one folder	cd ..	cd ..
Go to home folder	cd ~	cd %USERPROFILE%
Create a folder	mkdir my_project	mkdir my_project
Clear the screen	clear	cls
Stop a running app	Ctrl + C	Ctrl + C

Tip: folder names containing spaces should be wrapped in quotation marks.

Works on both macOS and Windows when you are already in the parent folder

```
cd "My AI Project"
```

3. Create and use an Anaconda environment

A separate environment keeps each project's Python version and packages isolated. Use one environment per project whenever possible.

Task	macOS Terminal	Windows (Anaconda Prompt)
Check Conda	<code>conda --version</code>	<code>conda --version</code>
List environments	<code>conda env list</code>	<code>conda env list</code>
Create environment	<code>conda create -n myenv python=3.11</code>	<code>conda create -n myenv python=3.11</code>
Activate environment	<code>conda activate myenv</code>	<code>conda activate myenv</code>
Check Python	<code>python --version</code>	<code>python --version</code>
Deactivate	<code>conda deactivate</code>	<code>conda deactivate</code>
Delete environment	<code>conda remove -n myenv --all</code>	<code>conda remove -n myenv --all</code>

How do you know it is active?

Look at the beginning of the prompt. If you see something like (myenv), that Conda environment is active.

Example on macOS

```
(myenv) nachiketh@MacBook my_project %
```

Example on Windows

```
(myenv) C:\Users\Learner\Desktop\my_project>
```

4. Install dependencies safely

After activating the environment, install packages into that environment. For beginners, this form is especially reliable:

```
python -m pip install pandas
python -m pip install fastapi uvicorn
```

If the project provides a requirements.txt file, install everything listed in it:

```
python -m pip install -r requirements.txt
```

Why use "python -m pip"?

It makes it clearer that pip belongs to the Python interpreter in your currently active environment. This reduces the chance of installing packages into the wrong Python.

5. Complete beginner workflow

Assume you downloaded a project named `my_project` to your Desktop and it contains `app.py` and `requirements.txt`.

macOS

```
cd ~/Desktop/my_project
ls
conda create -n myproject python=3.11
conda activate myproject
python --version
python -m pip install -r requirements.txt
python app.py
```

Windows - Anaconda Prompt

```
cd %USERPROFILE%\Desktop\my_project
dir
conda create -n myproject python=3.11
conda activate myproject
python --version
python -m pip install -r requirements.txt
python app.py
```

When the program is running

Keep that terminal window open. To stop the program, press `Ctrl + C`. When you are finished with the environment, run: `conda deactivate`

If python does not work on macOS

Inside a correctly activated Conda environment, `python` normally works on both systems. If you are running outside Conda and macOS reports that `python` is unavailable, try:

```
python3 app.py
```

Do not randomly switch between `python` and `python3` inside a project. First activate the intended Conda environment, then check `python --version`.

6. Run FastAPI

If your file is `main.py` and contains a FastAPI object named `app`, use:

```
conda activate myproject
python -m pip install fastapi uvicorn
python -m uvicorn main:app --reload
```

Then open these addresses in your browser:

```
Application:  http://127.0.0.1:8000
API docs:    http://127.0.0.1:8000/docs
```

Stop the server with `Ctrl + C`.

7. Run Jupyter Lab

```
conda activate myproject
python -m pip install jupyterlab
jupyter lab
```

Jupyter normally opens automatically in your browser. Keep the terminal running while you use Jupyter.

8. Clone and run a GitHub project

Once Git is installed, the basic flow is the same on both macOS and Windows:

```
git clone https://github.com/example/project.git
cd project
conda create -n projectenv python=3.11
conda activate projectenv
python -m pip install -r requirements.txt
python app.py
```

Important

The last command depends on the project. It may be `python app.py`, `python main.py`, `python -m uvicorn main:app --reload`, or another command stated in the project README.

9. Common beginner problems

"conda" is not recognized

Windows: open Anaconda Prompt instead of a normal Command Prompt. macOS: reopen Terminal after installing Anaconda/Miniconda. If needed, initialize the shell using the Conda installation instructions.

"python" is not recognized

Activate the intended Conda environment first. Then run `python --version`. On macOS outside Conda, `python3` may be the available command.

No such file / directory

You are probably in the wrong folder or the folder name is different. Use `ls` on macOS or `dir` on Windows to inspect what is actually there.

ModuleNotFoundError

Activate the correct environment, then install the missing package with `python -m pip install <package-name>`.

requirements.txt not found

Use `ls` or `dir`. Confirm that you are inside the project folder that actually contains `requirements.txt`.

Windows Desktop path fails

Some Windows systems store Desktop inside OneDrive. Try: `cd "%USERPROFILE%\OneDrive\Desktop\my_project"`

Port already in use

Another server may already be running. Stop it with `Ctrl + C` in its terminal, or close the previous terminal process before starting again.

Best debugging habit

Do not guess. First check: 1) Which folder am I in? 2) Which Conda environment is active? 3) Which Python version is running? 4) Are the required packages installed?

10. One-page command cheat sheet

These are the commands you will use most often during beginner labs.

Task	macOS Terminal	Windows (Anaconda Prompt)
Current folder	<code>pwd</code>	<code>cd</code>
List files	<code>ls</code>	<code>dir</code>
Enter folder	<code>cd folder_name</code>	<code>cd folder_name</code>
Back one level	<code>cd ..</code>	<code>cd ..</code>
Create env	<code>conda create -n myenv python=3.11</code>	<code>conda create -n myenv python=3.11</code>
Activate env	<code>conda activate myenv</code>	<code>conda activate myenv</code>
Install project	<code>python -m pip install -r requirements.txt</code>	<code>python -m pip install -r requirements.txt</code>
Run Python	<code>python app.py</code>	<code>python app.py</code>
Run FastAPI	<code>python -m uvicorn main:app -- reload</code>	<code>python -m uvicorn main:app -- reload</code>
Run Jupyter	<code>jupyter lab</code>	<code>jupyter lab</code>
Stop program	<code>Ctrl + C</code>	<code>Ctrl + C</code>
Deactivate env	<code>conda deactivate</code>	<code>conda deactivate</code>
Clear screen	<code>clear</code>	<code>cls</code>

The workflow to remember

1. Go to the project folder.
2. Create the environment once.
3. Activate the environment every time you start working.
4. Install dependencies.
5. Run the program.
6. Stop with Ctrl + C when needed.
7. Deactivate when finished.

The golden rule

When something fails, check your folder and active Conda environment before changing code.

Manifold AI Learning

Build confidently. One command at a time.